KATEDRA INFORMATIKY
UNIVERZITA PALACKÉHO V OLOMOUCI
www.inf.upol.cz

**Michal Krupka**
krupka.inf.upol.cz
michal.krupka@upol.cz
17. listopadu 12, 771 46 Olomouc

**Paradigmata programování 2 ⋄ poznámky k přednášce**

# 3. Více o makrech

verze z 2. března 2025

Minule jsme zavedli makra a ukázali, jak fungují ve vyhodnocovacím procesu. Dnes se soustředíme na komplikovanější příklady a na problémy, na které programátor během práce s makry narazí.

## 1 Co dosud víme o makrech

Makro je nástroj umožňující abstrakci na úrovni zdrojového kódu. Pomocí maker lze definovat operátory s vlastními pravidly vyhodnocování, podobně jako je tomu u speciálních operátorů. Někdy se říká, že tím makra umožňují přizpůsobit si programovací jazyk řešenému problému. V definici makra se zadává **expanzní funkce makra**, která převádí vyhodnocované makro-výrazy na jiné výrazy. Nové makro se definuje speciálním operátorem `defmac`.

```
(defmac when
        (lambda (test &rest expressions)
          `(if ,test (progn ,@expressions) nil)))
```

Makra jsou prvky jazyka, které nejsou hodnotami (podobně jako prostředí). Informace o tom, že symbol je názvem makra, je uložena v jeho globální funkční vazbě. Symbol tedy nemůže současně pojmenovávat makro a funkci. Pokus o použití operátoru `function` na název makra vede k chybě (stejně jako u speciálního operátoru).

Makro lze definovat také pomocí našeho makra `defmacro`:

```
(defmacro when (test &rest expressions)
  `(if ,test (progn ,@expressions) nil))
```

Expanzní funkci makra lze zjistit použitím funkce `macro-function` na název makra. S její pomocí jsme si napsali funkci `macroexpand-1`, kterou je vhodné používat při testování maker.

Expanze makra:

```
(when t 1) ⟶ (if t (progn 1) nil)
(when (> x y) 1) ⟶ (if (> x y) (progn 1) nil)
(when (> x y) (out z) (+ x y))
    ⟶ (if (> x y) (progn (out z) (+ x y)) nil)
```

Při vyhodnocování makro-výrazu se výraz po expanzi znovu předloží vyhodnocovacímu procesu k vyhodnocení.

Vyhodnocení makro-výrazu se tedy skládá ze dvou kroků. Při ladění je dobré zabývat se každým krokem zvlášť.

## 2 $\lambda$-seznam expanzní funkce: klíčové slovo `&body`

Víme, že $\lambda$-seznamy funkcí mohou obsahovat mimo jiné klíčová slova `&rest`, `&key` a `&optional`. U $\lambda$-seznamu expanzních funkcí maker je možné místo klíčového slova `&rest` použít `&body`.

Klíčové slovo `&body` má stejný význam, jako klíčové slovo `&rest`, ale ukazuje, že seznam parametrů, který je jím uvozen, tvoří tělo, tedy výrazy, které se budou vyhodnocovat. To ovlivňuje automatické formátování výrazů s makrem, jak ukazuje **příklad**:

Při definici makra `when` s $\lambda$-seznamem `(test &rest expressions)` bude formátování (např. pomocí tabelátoru) vypadat takto:

```
(when x
      (out 'jedna)
      'dvě)
```

Pokud použijeme $\lambda$-seznam `(test &body expressions)`, bude formátování lepší:

```
(when x
  (out 'jedna)
  'dvě)
```

To je jediný rozdíl mezi klíčovými slovy `&rest` a `&body`.

Ve zdrojovém kódu k této přednášce jsou některá makra, která jsme napsali minule s použitím klíčového slova `&rest`, přepsána se slovem `&body`. Prohlédněte si makra, která jste psali na minulém cvičení, a pokud je to vhodné, také je takto upravte.

# 3 Problém vícenásobného vyhodnocení

Makro `or` z minulého semestru vracelo jako hodnotu *Pravda* hodnotu posledního vyhodnocovaného výrazu, nikoli symbol `t`:

```
> (or 'a 'b)
A

> (or nil 'b)
B
```

To lze někdy s výhodou využít; představte si třeba, že chceme zjistit příjmení uživatele hypotetickou funkcí `surname` a pokud uživatel příjmení nemá (funkce `surname` vrátí `nil`), použít místo něj prázdný řetězec. To můžeme díky chování makra `or` udělat takto:

```
(or (surname user) "")
```

(Těžko najít jiný způsob, jak problém vyřešit podobně jednoduše. Na druhou stranu je trochu pochybné, zda je vhodné takto používat řetězce jako logické hodnoty; každopádně v Lispu je to možné.)

Chceme vylepšit naši verzi makra `or` pro dva argumenty z minula, aby měla popsané vlastnosti. Přijdeme s tímto řešením:

```
(defmacro or-2 (x y)
  `(if ,x ,x ,y))
```

Při testování nového makra zřejmě nenarazíme na nic zvláštního. Například testy uvedené před chvílí proběhnou podle očekávání. Přesto makro nepracuje správně. Podívejte:

```
> (or-2 (print 'a) nil)
A
A
A
```

Výraz `(out 'a)` se vyhodnotil dvakrát.

**Další příklad.** Uvažme následující testovací variantu makra `when`: makro `whenv` má pracovat stejně jako makro `when`, ale s tím rozdílem, že jako výsledek vrátí hodnotu testované podmínky:

```
> (whenv (cdr '(1 2))
    (print "Ano"))
"Ano"
(2)
```

Nejjednodušší možnost (a nesprávná, jak uvidíme za chvíli), jak makro napsat, je tato:

```
(defmacro whenv (test &body body)
  `(when ,test
     ,@body
     ,test))
```

Makro projde před chvílí uvedeným testem podle očekávání. Určité podezření bychom ale měli pojmout při expanzi makra. Testovaný výraz expanduje takto:

```
(when (cdr '(1 2))
  (print "Ano")
  (cdr '(1 2))))
```

Vidíme, že při vyhodnocení se výraz `(cdr '(1 2))` vyhodnotí dvakrát, což jistě není to, co by uživatel čekal. Vidět je to při vedlejším efektu:

```
> (whenv (cdr (print '(1 2)))
    (print "Ano"))
(1 2)
"Ano"
(1 2)
(2)
```

Zde bychom jistě čekali, že se seznam `(1 2)` vytiskne jen jednou.

Narazili jsme na první problém s makry, na který je třeba si dát pozor: **problém vícenásobného vyhodnocení**.

Problém demonstrujeme ještě na jednom příkladě. Chceme napsat makro `test-number`, které realizuje větvení na tři větve podle znaménka zadaného čísla:

```
(defmacro test-number (number minus zero plus)
  `(if (minusp ,number)
       ,minus
     (if (zerop ,number)
         ,zero
       ,plus)))
```

S makrem je zdánlivě všechno v pořádku, ale my už tušíme, že zadaný výraz bude vyhodnocovat vícekrát. Můžete si sami vyzkoušet, k čemu povede toto:

```
(let ((n 10))
  (test-number (print n)
    'minus
    'zero
    'plus))
```

# 4 Problém zabrání symbolu

Problém s vícenásobným vyhodnocováním např. u makra `whenv` se můžeme pokusit vyřešit pomocí nové vazby, na kterou navážeme vyhodnocený argument:

```
(defmacro whenv (test &body body)
  `(let ((result ,test))
     (when result
       ,@body
       result)))
```

Test:

```
> (whenv (cdr (print '(1 2)))
    (print "Ano"))
(1 2)
"Ano"
(2)
```

Vidíme, že ke dvojímu vyhodnocení testovaného výrazu nedošlo.

Tím jsme vyřešili problém vícenásobného vyhodnocení, ale bohužel vytvořili problém nový, a to **problém zabrání symbolu** (*symbol capture*). Zkusme tento test:

```
> (let ((result '(1 2)))
    (whenv (cdr result)
      (out "Neprázdné cdr v seznamu: " (strs result))))
Neprázdné cdr v seznamu: (2)
(2)
```

Vytištěný seznam `(2)` na předposledním řádku jsme nečekali. Měl to být seznam `(1 2)`.

Abychom zjistili, v čem je problém, nahradíme makro-výraz v testu jeho expanzí:

```
(let ((result '(1 2)))
  (let ((result (cdr result)))
    (when result
      (out "Neprázdné cdr v seznamu: " (strs result))
      result)))
```

Teď už by to mělo být jasné. Došlo k zabrání symbolu `result`.

K **zabrání symbolu** může dojít, pokud je v expanzi makra vytvořeno prostředí, které zastiňuje prostředí vytvořené uživatelem, a symboly v uživatelově kódu tak mají jiné než předpokládané aktuální vazby.

Problém lze řešit vygenerováním jedinečného symbolu pomocí funkce `gensym`. Funkci lze volat bez argumentu nebo s jedním argumentem. Pokud je argument použit, měl by to být řetězec a dostane se do názvu symbolu. Symboly vytvořené funkcí `gensym` začínají znaky „`#:`“.

```
> (gensym)
#:G1144

> (gensym "SYMBOL")
#:SYMBOL1145
```

Pokud ale symbol začínající „`#:`“ napíšeme, dostaneme vždy nový symbol téhož názvu:

```
> (eql * '#:SYMBOL1145)
NIL
```

Tím je zajištěna jedinečnost symbolů vytvořených funkcí `gensym`.

Třetí (a konečně správná) verze makra vypadá takto:

```
(defmacro whenv (test &body body)
  (let ((res-symbol (gensym)))
    `(let ((,res-symbol ,test))
       (when ,res-symbol
         ,@body
         ,res-symbol))))
```

Výraz

```
(whenv (cdr result)
  (out "Neprázdné cdr v seznamu: " (strs result)))
```

nyní expanduje na

```
(LET ((#:G1151 (CDR RESULT)))
  (WHEN #:G1151
    (OUT "Neprázdné cdr v seznamu: (STRS RESULT)")
    #:G1151))
```

Správná implementace makra `or-2`:

```
(defmacro or-2 (x y)
  (let ((x-symbol (gensym "X")))
    `(let ((,x-symbol ,x))
       (if ,x-symbol ,x-symbol ,y))))
```

Uvedený test už skončí správně:

```
> (or-2 (print 'a) nil)
A
A
```

Expanze testovaného výrazu:

```
(LET ((#:X782 (PRINT 'A)))
  (IF #:X782 #:X782 NIL))
```

Správná implementace makra `test-number`:

```
(defmacro test-number (number minus zero plus)
  (let ((value (gensym "VALUE")))
    `(let ((,value ,number))
       (if (minusp ,value)
           ,minus
         (if (zerop ,value)
             ,zero
           ,plus)))))
```

Využiji příležitosti a uvedu ještě jednu funkci, která vrací symbol: funkci `symbol`. Funkce vrací symbol, jehož jméno je zadáno jako textový řetězec:

```
> (symbol "DEFUN")
DEFUN
```

Funkce vrací symboly, které normálně používáme ve zdrojovém kódu. Vstupní řetězec by měl ze znaků obsahovat jen velká písmena.

Vraťme se k makrům. Makro `whenv` lze napsat i takto:

```
(defun whenv-help (cond-val body-fun)
  (when cond-val
    (funcall body-fun)
    cond-val))

(defmacro whenv (test &body body)
  `(whenv-help ,test (lambda () ,@body)))
```

Toto řešení má jistě své výhody: nezpůsobuje problém vícenásobného vyhodnocení a zabrání symbolu a nepoužívá funkci `gensym`. Používá také výhodnou strategii psaní maker: neřešit vše v expanzi výrazu, ale expandovat výraz na co nejjednodušší aplikaci pomocné funkce. Pracovně tomu říkáme **pravidlo co nejrychlejšího úniku z makra do funkce**.

Na závěr ještě poznamenejme, že makro `whenb` z minulé přednášky také zabírá symbol, ale v tomto případě jde o jeho zamýšlený efekt.

**Hlavní problémy s makry**

- vícenásobné vyhodnocení
- zabrání symbolu
- jsou to neprominutelné chyby

# 5 Rekurze v makrech

Složitější makra ve své expanzi používají samy sebe. Takovým makrům se říká *rekurzivní makra*. Příkladem může být tato verze makra `and` pro libovolný počet argumentů:

```
(defmacro and (&rest forms)
  (if forms
      `(when ,(car forms) (and ,@(cdr forms)))
    t))
```

(Makro stále není ekvivalentní makru `and` z minulého semestru; v případně kladného výsledku vrací vždy `t` místo konkrétnější hodnoty.)

Další příklad je, jak uvidíme za chvíli, problematický:

```
(defmacro while (test &body body)
  `(when ,test
     ,@body
     (while ,test ,@body)))
```

Abychom pochopili, v čem tkví problém tohoto příkladu, musíme si stručně říci o kompilaci zdrojového kódu.

*PP Polyglot* obsahuje **kompilátor** (překladač) zdrojového kódu. Ten

- převádí program napsaný v Lispu na ekvivalentní program napsaný ve strojovém kódu procesoru.

To má za následek mnohem rychlejší výpočet a také podstatnou úsporu paměti (mimo jiné kompilátor umí rozpoznat koncově rekurzivní funkce a zkompilovat je tak, že nezatěžují zásobník, jak jsme viděli v minulém semestru).

*PP Polyglot* používá překladač v editoru. Kdykoli v něm vyhodnocujeme nějaký výraz, aplikace jej nejprve přeloží a teprve potom jej začne vyhodnocovat. Přeloží také všechny funkce definované operátorem `defun`.

Výrazy v listeneru se nepřekládají a jsou vyhodnocovány přesně postupem, který jsme nazvali vyhodnocovacím procesem a na minulé přednášce jsme doplnili.

Jelikož překlad vytváří strojový kód ekvivalentní původnímu kódu v Lispu, stále to vypadá, že se výrazy stále vyhodnocují podle pravidel vyhodnocovacího procesu, i když skutečnost je jiná. (Zkompilovaný kód není založen na zkoumání složených výrazů a jejich podvýrazů.)

Tento fakt má jednu důležitou výjimku.

- Aby mohl kompilátor zdrojový kód přeložit, **expanduje všechna použitá makra**.

Při použití kompilátoru tedy vyhodnocovací proces pracuje jinak, než jak jsme ho popsali minule: ve vyhodnocovaném složeném výrazu jsou nejprve expandována makra, a to pouze poprvé (při jeho opětovném vyhodnocování se už znovu neexpandují).

To většinou vede ke stejným výsledkům, ale s jednou důležitou výjimkou: rekurzivními makry. Pro demonstraci si vytvořme testovací verzi makra `when`, která bude při expanzi tisknout zprávu:

```
(defmacro test-when (test &rest expressions)
  (out "Expanding TEST-WHEN with test: "
       (strs test)
       " and expressions: "
       (strs expressions))
  `(if ,test (progn ,@expressions) nil))
```

a definujme následující funkci:

```
(defun compilation-test ()
  (test-when (< 0 1) 'ano))
```

Nyní můžete porovnat, co se děje, když definici vyhodnotíte v listeneru a pak ji v listeneru zkusíte funkci. Potom zkuste definici vyhodnotit v editoru (kde se zkompiluje), podívat se, co se při definici vytiskne, a pak funkci zavolat.

Při kompilaci výrazu obsahujícího makro `while` se kompilátor zacyklí, protože se pokusí expandovat všechna makra. Můžete to vyzkoušet třeba definicí této funkce (v editoru a listeneru):

```
(defun while-test ()
  (let ((n 10))
    (while (> n 0)
      (print n)
      (setf n (- n 1)))))
```

Je několik způsobů, jak makro `while` napsat správně. Jeden z nich je tento:

```
(defmacro while (test &body body)
  `(do-while (lambda () ,test)
             (lambda () ,@body)))

(defun do-while (test-fun body-fun)
  (when (funcall test-fun)
    (funcall body-fun)
    (do-while test-fun body-fun)))
```

Makro teď používá pomocnou funkci `do-while`. Ta je rekurzivní, což ovšem u funkce samozřejmě nevadí. (Všimněte si, že je opět použité pravidlo co nejrychlejšího úniku z makra do funkce.)

Nyní můžete funkci `while-test` znovu zkusit definovat v listeneru i editoru.

**Pravidlo psaní rekurzivních maker**

Ukončení rekurze u rekurzivních maker musí testovat expanzní funkce (v čase expanze).

Makro `and` napsané na začátku této části tuto podmínku splňuje, makro `while` nikoli.

Jako složitější příklad rekurzivního makra je ve zdrojovém kódu k této přednášce napsáno makro `cond` a pomocí něj makro `case`.

# 6 Čas expanze a čas běhu

Makro `case` je podobné příkazu `switch` v jazyce C. Jak pracuje, pochopíme z příkladu:

```
(case x
  (1 "jedna")
  (2 "dvě")
  (3 "tři"))
```

Chceme napsat jeho variantu `random-case`, která nebude obsahovat testovanou hodnotu, ale větev, která se vykoná, vybere náhodně. Například výraz

```
(random-case
  ("jedna")
  ("dvě")
  ("tři"))
```

by měl náhodně vrátit jeden z řetězců `"jedna"`, `"dvě"` nebo `"tři"`. (Řetězce jsou v závorkách, aby větve mohly obsahovat víc výrazů.)

Zde jsou tři možnosti, jak makro napsat. Jedna z nich je nesprávná, jedna nešikovná a jedna optimální:

```
(defun random-case-help (body)
  (if (null body)
      '()
    (cons (cons (- (length body) 1) (car body))
          (random-case-help (cdr body)))))

(defmacro random-case (&body body)
  `(case (random (length ',body))
     ,@(random-case-help body)))
```

```
(defmacro random-case (&body body)
  `(case (random ,(length body))
     ,@(random-case-help body)))

(defmacro random-case (&body body)
  `(case ,(random (length body))
     ,@(random-case-help body)))
```

Rozdíl mezi těmito možnostmi je samozřejmě v tom, za jakých okolností se které výrazy vyhodnocují: zda **v čase expanze makra** (tj. jsou vyhodnocované expanzní funkcí), nebo **v čase běhu** (tj. jsou pouze expanzní funkcí vytvářené a vracené jako její výsledek). Pohledem do jejich zdrojového kódu byste měli přijít na to, jaký je mezi nimi přesně rozdíl a jaké to bude mít důsledky.

Všechny tři možnosti si také vyzkoušejte prakticky a také si všimněte, že fungují různě, pokud běží v interpretovaném, nebo zkompilovaném kódu.

Řešení můžeme otestovat takto: napíšeme funkci

```
(defun test ()
  (random-case
    ("jedna")
    ("dvě")
    ("tři")))
```

Postupně v ní vyzkoušíme všechny tři verze makra a také definici vždy jednak vyhodnotíme a jednak zkompilujeme. Postupně tak dostaneme šest verzí funkce, kterou vždy otestujeme tak, že ji několikrát zavoláme.

# Nové symboly v jazyce

## Vestavěné

**funkce:** `gensym`, `symbol`

**další:** `&body`

## Definované na přednášce (ve zdrojovém kódu)

**makra:** `cond`, `case`

## Definované na cvičení

**makra:** `prog1`, `and`, `or`, `let*`

## Otázky a úkoly na cvičení

1. Napište makro `awhen`, které je obdobou makra `when`, ale v jeho těle je proměnná `it` navázána na hodnotu podmínky:

```
> (awhen (cdr '(a b))
    (print it)
    nil)
(B)
NIL
```

2. Napište makro `setq-seq-2`, které vyhodnotí svůj první argument, do druhého (který musí být symbol) hodnotu uloží a do třetího (který musí být rovněž symbol) ji uloží zvětšenou o 1:

```
> (let ((a 'a) (b 'b))
    (setq-seq-2 (+ 1 1) a b)
    (list a b))
(2 3)
```

3. Napište makro `error-if-nil`, které vyvolá chybu, když zadaný výraz má hodnotu `nil`, a jinak jeho hodnotu vrátí. V chybové zprávě navíc uvede výraz, u kterého k chybě došlo (chybová hláška je přibližná):

```
> (+ 1 (error-if-nil (car (list 2))))
3

> (+ 1 (error-if-nil (cdr (list 2))))
Hodnota výrazu (CDR (LIST 2)) je nil.
```

4. Napište makro `do-interval`, které projde všechna celá čísla v zadaném intervalu a pro každé vyhodnotí výrazy svého těla. Interval je zadán dolním a horním koncem:

```
> (do-interval (x 5 (* 5 2))
    (print x))
5
6
7
8
9
10
NIL
```

Makro zkuste implementovat co nejrychlejším únikem do funkce.

5. Vysvětlete poslední uvedenou verzi makra `whenv`.

6. Přepište podobným způsobem makro `test-number`.

7. Makro `prog1` by mělo fungovat stejně jako operátor `progn`, ale na rozdíl od něj by mělo vracet hodnotu prvního výrazu svého těla. Následují tři pokusy o jeho implementaci. První má nevýhodu, že potřebuje pomocnou funkci a druhý, že definuje proměnnou, kterou nepotřebuje. Rozhodněte, ve kterém z řešení navíc dochází k problému zabrání symbolu. Pokud ano, uveďte vždy příklad, pokud ne, zdůvodněte.

```
(defmacro prog1-1 (first &body body)
  `(let ((result ,first)
         (fun (lambda () ,@body)))
     (funcall fun)
     result))

(defmacro prog1-2 (first &body body)
  `(let ((result ,first)
         (bresult (progn ,@body)))
     result))

(defmacro prog1-3 (first &body body)
  `(let ((result ,first))
     ,@body
     result))
```

8. Napište vlastní verzi makra `prog1`. *(do jazyka)*

9. Odhadněte a pak vyzkoušejte, na co expandují následující výrazy s užitím makra `and` z přednášky:

```
(and)
(and a)
(and a b)
(and a b c)
```

10. Vylepšete makro `and` tak, aby místo `t` vracelo hodnotu posledního výrazu. Pro výraz `(and)` by mělo stále vracet `t`. (Lze to udělat jednoduše pomocí `&optional`-parametru, který můžeme uvést před `&rest`-parametrem.) *(do jazyka)*

11. Co říkáte na tuto variantu makra `and`?

```
(defmacro my-and (&rest args)
  `(cond ((null ',args) t)
         (,(car args) (my-and ,@(cdr args)))
         (t nil)))
```

12. Napište makro `or` pro libovolný počet argumentů. V první fázi ho můžete napsat tak, aby vracelo jen `t` nebo `nil`, pak ho vylepšete, aby vracelo hodnotu posledního vyhodnoceného výrazu, jako to dělá operátor `or` z minulého semestru. *(do jazyka)*

13. Napište makro `defun-doc`, které kromě definování nové funkce stejně jako operátor `defun` vytiskne i její název, $\lambda$-seznam a dokumentaci (nakonec vrátí její název jako operátor `defun`). Příklad:

```
> (defun-doc addition (a b)
    "Sčítání dvou čísel"
    (+ a b))
Function: ADDITION
Lambda-list: (A B)
Documentation: Sčítání dvou čísel
ADDITION
```

14. Zobecněte makro `setq-seq-2` na makro `setq-seq`, které bude přijímat libovolný počet proměnných s tím, že každou následující nastaví na hodnotu o jedna větší, než předchozí:

```
> (let ((a 'a) (b 'b) (c 'c) (d 'd))
    (setq-seq (+ 1 1) a b c d)
    (list a b c d))
(2 3 4 5)
```

15. Napište makro `let*`, které bude fungovat stejně jako stejnojmenný operátor z minulého semestru. *(do jazyka)*

16. Napište makro `all-cond`, které pracuje podobně jako makro `cond`, ale vyhodnotí všechny větve, jejichž podmínka je splněna. Následující výraz:

```
(all-cond ((< 1 2) (print 1))
          ((> 1 2) (print 2))
          ((= 1 1) (print 3))
          ((= 1 2) (print 4)))
```

tedy vytiskne postupně čísla `1` a `3`. Návratovou hodnotou makra se nezabývejte. Makro lze snadno napsat bez použití rekurze.

17. Následující příklady jsou těžší. První z nich: Napište makro `op-alias` sloužící k definici nového jména zadaného operátoru (např. kvůli zkrácení dlouhého názvu). Například po vyhodnocení výrazu

```
(op-alias funcall fc)
```

bychom mohli místo operátoru `funcall` v libovolném výrazu použít `fc`:

```
> (fc #'list 1 2 3)
(1 2 3)
```

Makro musí fungovat správně na všechny typy operátorů (tj. funkce makra i speciální operátory). Nový operátor (v uvedené ukázce operátor `fc`) nemusí být funkce.

18. Při řešení problému zabrání symbolu píšeme pokaždé podobný kód. Napište makro `with-gensyms`, které nám tuto práci zjednoduší takto:

    Když definujeme např. makro `or-2` z přednášky takto:

```
(defmacro or-2 (x y)
  (with-gensyms ((x-symbol x))
    `(if ,x-symbol ,x-symbol ,y)))
```

výraz `(or-2 a b)` expanduje na něco jako

```
(let ((#:G1483 a))
  (if #:G1483 #:G1483 b))
```

První argument makra má stejný tvar jako první argument makra `let` a může obsahovat více prvků než jeden.

19. Někdy se používá makro `rebinding`, které někdy zjednoduší psaní maker ještě více. Po definici

```
(defmacro or-2 (x y)
  (rebinding (x)
    `(if ,x ,x ,y)))
```

výraz `(or-2 a b)` expanduje podobně jako v předchozím příkladě. Seznam, který je prvním argumentem makra `rebinding`, může obsahovat libovolný počet proměnných.